



IILM UNIVERSITY

Industrial Internship Report

on

PASSWORD MANAGER

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering

Submitted By

Aayush Raj

Roll Number: 2410030950

Section: 3CSE23

Batch: 2024-28

Internship Domain: Python Programming

Internship Duration: 4 Weeks

Internship Period: 04 August 2026 - 05 September 2026

In collaboration with

Upskill Campus, The IoT Academy and UniConverge Technologies (UCT)

CANDIDATE'S DECLARATION

I, **Aayush Raj**, hereby declare that the internship report titled "**Password Manager**" has been prepared by me to fulfil the academic requirements of the Bachelor of Technology in Computer Science and Engineering at IILM University, Greater Noida. The report presents the work and learning completed during my Python Programming internship. The project implementation and technical discussion have been documented to the best of my knowledge. I understand that I am responsible for the originality and correctness of the submitted material.

Signature

Student Name: Aayush Raj

Roll No.: 2410030950

ACKNOWLEDGEMENT

I sincerely express my gratitude to IILM University, Upskill Campus, The IoT Academy and UniConverge Technologies for providing the internship opportunity and a structured platform for practical learning. I am thankful for the learning resources, project guidance and technical exposure that helped me improve my understanding of Python programming.

I am also grateful to my mentors, faculty members, friends and family for their support, encouragement and constructive suggestions throughout the internship. The experience of learning concepts week by week and applying them to a working project helped me gain confidence in programming, debugging and project documentation.

Signature

Student Name: Aayush Raj

Roll No.: 2410030950

TABLE OF CONTENTS

Chapter	Chapter Name	Page
1	Candidate's Declaration	
2	Acknowledgement	ii
3	Internship Completion Certificate	1
4	Project Description	2
4.1	Introduction	2
4.2	Organization / Internship Profile	2
4.3	Problem Statement	2
4.4	Project Objectives	2
4.5	Scope of the Project	2
4.6	Technologies and Tools Used	3
4.7	System Architecture	3
4.8	Methodology	4
4.9	Expected / Achieved Outcomes	4
4.10	Project Implementation	4
5	Testing and Results	6
6	My Learnings	7
7	Future Work Scope	8
8	Bibliography / References	9

3. INTERNSHIP COMPLETION CERTIFICATE

The Internship Completion Certificate issued to Aayush Raj is reproduced below as evidence of completion of the internship. The certificate states the domain as **Python**, names Aayush Raj as the recipient, and records the internship period as **04-08-2026 to 05-09-2026**. It also identifies collaboration with AICTE, UniConverge Technologies and upSkill Campus.

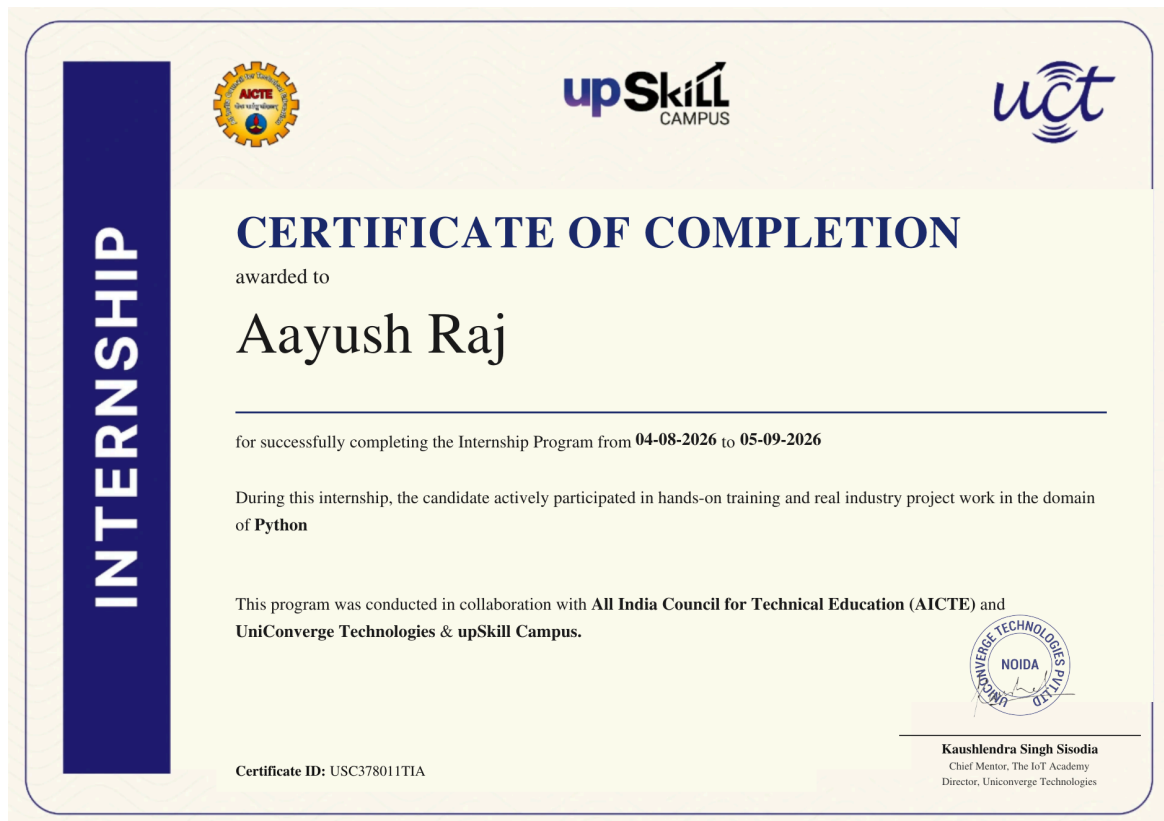


Fig. 3.1: Internship Completion Certificate

4. PROJECT DESCRIPTION

4.1 Introduction

The Password Manager is a Python-based command-line application developed as a practical learning project during the internship. The application is designed to help a user maintain website credentials in one local data store instead of keeping individual credentials in scattered notes. It demonstrates how Python functions, loops, conditional statements, dictionaries, file handling and JSON serialization can be combined to form a complete application workflow.

The current implementation focuses on simplicity and clarity. A menu is displayed when the program starts, and the user can add a password, view saved passwords or exit. The program uses a JSON file named **passwords.json** for persistent storage.

4.2 Organization / Internship Profile

The internship was undertaken through the Upskill Campus learning ecosystem in association with The IoT Academy and UniConverge Technologies. The internship structure combines guided learning activities, knowledge assessments and practical project work. During the internship, I progressed through Python learning modules and applied the acquired concepts to the Password Manager project.

4.3 Problem Statement

Users may need to remember credentials for many online services. Managing these credentials manually can become inconvenient and difficult to organize. The problem addressed by this project is to create a small, easy-to-use application that accepts website credentials, stores them in a structured local file and allows the user to retrieve them through a menu-driven interface.

4.4 Project Objectives

- Develop a functional Password Manager using Python.
- Apply Python functions, conditions, loops and file handling.
- Store credentials in a structured JSON format.
- Provide simple add and view operations through a command-line interface.
- Gain experience in testing, debugging and publishing code on GitHub.

4.5 Scope of the Project

The scope of the current project covers local credential entry, JSON-based persistent storage and retrieval through a command-line interface. The project is intentionally limited to a basic educational prototype. Advanced security controls such as encryption, master-password authentication, protected key storage and secure remote synchronization are not included in the current version and are identified as future enhancements.

4.6 Technologies and Tools Used

Component	Technology / Tool	Purpose
Programming Language	Python	Application logic and user interaction
Data Format	JSON	Local structured storage
Standard Libraries	json, os	Data serialization and file checking
IDE	Visual Studio Code	Editing, debugging and execution
Version Control	GitHub	Source-code hosting and submission

Table 4.1: Technology and Tool Stack

4.7 System Architecture

The system follows a simple sequence: user input -> menu selection -> function execution -> JSON storage/retrieval -> console output. The design separates the major operations into functions, which improves readability and makes the program easier to test and extend.

User	Menu	Processing	Storage	Output
Enters choice	1 / 2 / 3	Add / View / Exit	passwords.json	Result message / credentials

Fig. 4.1: High-level system flow

4.8 Methodology

Step 1 - Requirement Understanding: Identify the need for a simple local credential management application and define the main operations.

Step 2 - Design: Divide the application into functions for loading data, saving data, adding credentials and viewing credentials.

Step 3 - Implementation: Use Python dictionaries for in-memory data and the json module to persist data to a local file.

Step 4 - User Interaction: Create a continuous menu using a loop and use conditional statements to perform the selected operation.

Step 5 - Testing: Run the application with valid and invalid inputs and confirm that the expected messages and operations occur.

Step 6 - Version Control: Upload the source code to a public GitHub repository for project submission.

4.9 Expected / Achieved Outcomes

The project achieved its basic functional objectives. The application starts correctly, displays the Password Manager menu, accepts a website, username and password for a new entry, stores the entry in JSON format, and displays saved records when requested. The program also handles an invalid menu choice without crashing, as demonstrated during execution testing.

4.10 Project Implementation

The implementation uses two helper functions for storage. **load_passwords()** checks whether the JSON file exists and loads its content when available. **save_passwords(data)** writes the updated dictionary to the file. The **add_password()** function collects website, username and password values and inserts them into the dictionary before saving. The **view_passwords()** function reads the stored records and prints them to the console.

The main program loop displays three options: Add Password, View Passwords and Exit. A user selection is processed using conditional statements. Any selection other than the supported options results in an invalid-choice message and the menu is displayed again.

Representative Code Section

```
import json
import os

FILE = "passwords.json"

def load_passwords():
    if os.path.exists(FILE):
        with open(FILE, "r") as f:
            return json.load(f)
    return {}

def save_passwords(data):
    with open(FILE, "w") as f:
        json.dump(data, f, indent=4)
```

Fig. 4.2: Core JSON storage implementation

4.11 GitHub Project Submission

The source code has been published in the GitHub repository created for the internship. The repository contains the main Python implementation file **PasswordManager.py**. GitHub provides a versioned and accessible location for reviewing the submitted source code.

Repository Link: <https://github.com/aayushraj-45/upskillcampus>

Project Code: <https://github.com/aayushraj-45/upskillcampus/blob/main/PasswordManager.py>

4.12 Project Working

When the application starts, it presents the following menu: Add Password, View Passwords and Exit. Selecting Add Password requests the website, username and password. The record is stored under the website key in the JSON file. Selecting View Passwords loads the saved data and displays each stored record. Selecting Exit ends the program. Invalid selections are rejected and the menu continues to display.

4.13 Security Consideration

The present implementation is a learning prototype. Credentials are stored as plain text in the local JSON file, and the application does not yet provide encryption or master-password authentication. For this reason, the implementation should not be used to store real sensitive credentials without security improvements. This limitation is documented so that the report accurately reflects the current implementation.

5. TESTING AND RESULTS

Testing was performed by executing the Python program from Visual Studio Code and checking the behavior of each menu option. The focus was on functional correctness, input handling and persistence of stored records.

Test ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	Application start	Run PasswordManager.py	Main menu appears	PASS
TC-02	Add credential	Choose 1 and enter details	Credential saved	PASS
TC-03	View credentials	Choose 2	Saved credentials displayed	PASS
TC-04	Empty storage	Choose 2 before adding data	No passwords saved message	PASS
TC-05	Exit	Choose 3	Application terminates	PASS
TC-06	Invalid option	Enter an unsupported value	Invalid Choice message	PASS

Table 5.1: Functional test cases

5.1 Test Procedure

The program was run in the VS Code terminal. First, the menu was checked for correct display. Next, a valid add-password flow was used to verify input collection and data saving. The view option was then used to verify retrieval. Finally, invalid numeric/text input was entered to confirm that the program reported an invalid choice rather than terminating unexpectedly.

5.2 Results

The core test scenarios passed. The application produced the expected menu, accepted credential information, used JSON storage, retrieved saved records and handled invalid menu selection. For the current small local dataset, no noticeable performance issue was observed during normal use.

5.3 Limitations of Testing

Testing was performed on a small educational implementation and is not a formal security audit. No claims are made about production-level confidentiality, encryption strength or resistance to attack. Larger datasets, concurrent users, database storage and encrypted credentials would require additional testing in a future version.

6. MY LEARNINGS

This internship helped me move from learning isolated Python concepts to applying them in a small working application. I improved my understanding of how program structure, data storage and user interaction fit together to solve a practical problem.

6.1 Technical Learning

- Python syntax, variables, functions and control flow.
- Conditional statements and loops for menu-driven applications.
- Dictionaries for structured in-memory data.
- File handling using read and write operations.
- JSON serialization and deserialization.
- Basic error/input handling and debugging.
- GitHub repository management and source-code sharing.

6.2 Professional Learning

I learned the importance of dividing a problem into smaller tasks, testing each feature, documenting limitations honestly and maintaining a readable code structure. I also gained experience in organizing an internship report and linking technical work with a project submission.

6.3 Career Growth

The experience has strengthened my interest in Python development and given me a foundation for learning more advanced areas such as databases, application security, APIs and graphical user interfaces. The ability to convert a requirement into a working prototype will be useful in future academic and professional projects.

7. FUTURE WORK SCOPE

Several improvements can be implemented to transform the current educational prototype into a more complete password-management application.

7.1 Security Enhancements

- Add encryption for stored credentials.
- Introduce a master password and authentication mechanism.
- Securely manage encryption keys rather than storing them in plain text.
- Mask password input and avoid exposing credentials during normal display.

7.2 Functional Enhancements

- Add strong random password generation.
- Add search, edit and delete functionality.
- Add password-strength evaluation.
- Add categories, notes and timestamps for each account.
- Add import/export features where appropriate.

7.3 Engineering Enhancements

- Replace JSON storage with SQLite or another database when the dataset grows.
- Add automated unit and integration tests.
- Build a graphical or web interface.
- Add better exception handling and logging.
- Package the application for easier installation and use.

7.4 Future Validation

Future versions should be tested for confidentiality, integrity, recoverability, performance with larger datasets, usability and secure handling of invalid or unexpected input.

8. BIBLIOGRAPHY / REFERENCES

1. Python Software Foundation, Python Documentation and Standard Library documentation.
2. Python json module documentation and examples.
3. GitHub Documentation - repositories, commits and source-code hosting.
4. Upskill Campus / The IoT Academy internship learning resources and project instructions.
5. UniConverge Technologies (UCT) internship ecosystem and project guidance.
6. IILM University academic internship report format used as the structure reference for this report.

DECLARATION OF TECHNICAL ACCURACY

The report describes the Password Manager prototype according to the implementation submitted in the GitHub repository at the time of report preparation. Features such as encryption and authentication are described as future work because they are not part of the current codebase.